Fountain View Hall

Release 0.2.0

Derek R. Neilson

Jun 07, 2026

CONTENTS:

Fountain View Hall's management tool `fvh-manager` is used to mange reservations at Fountain View Hall

CHAPTER

ONE

USER GUIDE

This guide will walk you through `fvh-manager` and covers topics such as how to install and use fvh-manager.

1.1 Installation

1.1.1 User Prerequisites

To verify you meet all prerequisites, run:

```
uv --version
```

Expected output:

```
uv 0.x.x
```

Not:

```
command not found: uv
```

Important

Make sure the `uv` command runs before running the program.

If `uv --version` fails, use this guide to install uv.

1.1.2 Minimal Getting Started

Make sure you run these commands from the project directory

Run this once:

```
uv sync
```

Then run this to start on every subsequent run:

```
uv run fvh-manager
```

1.1.3 Activate Completion by Operating System and Shell

If desired, you can activate shell completions by running one of the following commands, depending on your operating system and shell:

Operating system	Shell	Activation command
Linux/macOS	Bash	`source completions/fvh-manager.bash`
Linux/macOS	Zsh	`source completions/_fvh-manager`
Linux/macOS	Fish	`source completions/fvh-manager.fish`
Windows	PowerShell	`. .\completions\fvh-manager.ps1`

Once shell completions are activated, make sure your virtual environment is active. You can then run the program with:

```
fvh-manager
```

To view available completions, type:

```
fvh-manager <tab>
```

1.2 Developer Installation

Meet all previous *prerequisites* and additional ones as specified bellow.

Install developer dependencies for python:

```
uv sync --all-groups
```

Verify that `pdflatex` is installed:

```
pdflatex -v
```

Verify that `git` is installed:

```
git -v
```

Verify that `lefthook` is installed:

```
lefthook -v
```

CHAPTER

TWO

REFERENCES

Name: Derek R. Neilson Description: This is the entrypoint and should is left light by design.

Info on Type. Typer is a command line interface (CLI) tool that will be used thought this project. It defines some decorators `@app.command` see https://typer.tiangolo.com for details.

`src.main.main()`

Return type
None

CHAPTER

THREE

DESIGN

The design process has been documented quite thoroughly but not all decisions are documented

3.1 References Used to Make This Project

ReStructuredText guides were used as references for this project (see footnote).[1]

> **Note**
>
> I had zero rst experience before reading these guides they where very helpful.

The Typer home page was used for learning about Typer and was integral to getting the CLI up.

SQLAlchemy is another tool I never used before. Its own documentation was sufficient for my usage.

3.2 Overview for Fountain View Hall

3.2.1 Overview

Fountain View Hall wants a program that tracks event attendance and projected revenue for multiple bookings.

3.2.1.1 Program Requirements

- Repeatedly prompt for event name, guest count, and estimated revenue
- Continue until the user enters “done”
- Track total events processed
- Track total guests
- Track total projected revenue
- Calculate the average guests per event
- Determine the largest event

[1] reStructuredText guides: reStructuredText Markup Specification, reStructuredText Primer, reStructuredText Primer by Material for Sphinx, and reStructuredText Primer by Sphinx Note: they all are missing some info that others have except for reStructuredText Markup Specification which is very dense and lacks rendered examples.

3.2.1.2 Business Rules

- Pricing Rules
  - Large events receive a discount
- Operational Rules
  - Every booking must include a guest count
  - Booking total must be calculated before the discount is applied
  - Revenue totals should be tracked

3.2.2 Planed Technology Usage

I planned on using various technologies throughout this project some are very necessary others save time. Each technology will be categorized and described below:

3.2.2.1 Editing/Developer Tools

- *Git* and *GitHub* as a VCS (version control system).[1]
- uv for package, project, and environment management.
- *lefthook* as a Git hook manager.
- *pre-commit* to run pre-commit scripts.[2]
  - *ruff* for Python linting and formatting running as a pre-commit script.
- *Sphinx* to help build this beautiful documentation.
- *pytest* as a testing suite.

3.2.2.2 Python Packages

- *typer* for command line interface completions and argument parsing.
- *SQLite* as a database[3]
- *SQLAlchemy* for object relational mapping (ORM)

3.3 Fountain View Hall Pseudocode and User Journeys

3.3.1 Problem Statement

The project must meet all *requirements*

[1] Go to the project repo to see commit history.
[2] All scripts (in scripts directory) are written for unix systems and use the bash language.
[3] See SQLite's home page for more details.

PYTHON MODULE INDEX

S

INDEX

M

S